

Learning Deep Convolutional Networks for Demosaicing

Minh Vu

Table of content

1. Introduction.....	2
2. The proposed model.....	3
2.1 Theoretical Background	3
2.2 Personal implementation	5
3. Results and Discussions.....	7
4. Conclusions	13
References	15

1. Introduction

Most digital cameras contain sensor arrays covered by a Color Filter Array (CFA) to capture light information [1]. Because of this hardware design, each pixel sensor only records partial spectral information (typically, one of the three primary colors: red, green, or blue). To produce a high-quality, full-color image, a computational process known as demosaicing is required to infer the two missing color values for every pixel. However, since demosaicing involves predicting missing data, it inevitably introduces errors that reveal visual artifacts in the reconstructed image. Common examples include "zipper effects," which are abrupt intensity of changes between neighboring pixels, and "false color artifacts," which are colors that were not present in the original scene. [1]

The Bayer filter is currently the most popular CFA used in both commercial manufacturing and academic research. Despite its dominance, it has been shown to have poor mathematical properties when analyzed in the frequency domain. Traditionally, demosaicing algorithms have been divided into two groups: interpolation-based and dictionary-based methods. Interpolation-based methods rely on "handcrafted" mathematical rules derived from local observations. While these methods are fast, they often fail to accurately reconstruct complex structures and are limited by the rules designed by humans. In contrast, dictionary-based methods use libraries of learned image patches to represent local distributions more faithfully, offering better color quality. However, these approaches require expensive online optimization, making them too slow for many practical, real-world applications. [1]

Researchers have explored alternative CFA designs that are theoretically superior to the Bayer filter. However, these complex designs require effective demosaicing methods to fully release their potential. Because these custom methods are difficult to design by hand, significantly less effort has been put into developing demosaicing algorithms for non-Bayer patterns. To address these challenges, there is a need for a flexible, data-driven approach. Convolutional Neural Networks (CNNs) provide an ideal solution because they can automatically learn appropriate features and high-level relationships between samples. By designing a CNN for demosaicing, we can create a powerful tool that is both practical for real-world use and easily adaptable to any CFA design. [1] The next chapter presents the DeMosaicing Convolutional Neural Network (DMCNN), proposed by Nai-Sheng Syu, Yu-Sheng Chen, and Yung-Yu Chuang from the National Taiwan University. Besides that, several adaptations to the model's architecture to make it more suitable as a personal project are also discussed. Chapter III discusses the results of the model together with

some discussion, and Chapter IV concludes the project with the potential of this model and CNN in demosaicing in general.

2. The proposed model

This chapter starts with how Nai-Sheng Syu, Yu-Sheng Chen, and Yung-Yu Chuang implemented the DMCNN model. Then, it will introduce the personal implementation version of this project.

2.1 Theoretical Background

According to Nai-Sheng Syu et al., the DeMosaicing Convolutional Neural Network (DMCNN) is a deep learning model designed to perform end-to-end demosaicing by learning the mapping between mosaic samples and full-color image patches. The architecture is primarily inspired by the Super Resolution Convolutional Neural Network model. Unlike traditional methods that use fixed mathematical rules, DMCNN uses a data-driven approach where features are automatically explored and optimized during the training process. [1]

The model receives 33x33-pixel mosaiced patches (which are densely extracted from the training images) as input. Each pixel of the patch consists of three color channels, with the two missing channels blank. Thus, the input size is 33x33x3, allowing a unified network to handle different pixel locations in the Bayer pattern without needing separate models. If patches of size 33x33x1 are used, four separate networks could be necessary for the four different locations in the Bayer pattern. Furthermore, this structure makes the network flexible enough to be adapted for any other color filter array (CFA) design, which was unfortunately not pursued in this project.

The DMCNN architecture consists of three specialized layers:

1. **Feature Extraction Layer:** This layer uses 128 filters of size 9x9 to extract useful local features from the input patch. The filters are initialized as Gaussian kernels, and the output is a high-dimensional feature map.
2. **Non-linear Mapping Layer:** The second layer uses 64 filters of size 1x1. Its purpose is to map the high-dimensional feature vectors into a lower-dimensional representation. By using 1x1 kernels, the network performs this mapping pixel-by-pixel without considering the surrounding neighbors at this stage.
3. **Reconstruction Layer:** The final layer uses 5x5 kernels to reconstruct the full-color patch. The filters were also initialized as Gaussian kernels, and this layer exploits local information from the feature maps to produce the final RGB values.

The network uses Rectified Linear Units (ReLU) as the activation function between layers to prevent problems like vanishing or exploding gradients. Mathematically, the network can be formulated as:

$$\begin{aligned} F_1(Y) &= \max(0, W_1 * Y + B_1), \\ F_2(Y) &= \max(0, W_2 * F_1(Y) + B_2), \\ F(Y) &= W_3 * F_2(Y) + B_3 \end{aligned}$$

Where Y is the input patch, W represents the filters, B is the bias, and F_i represents the output feature map of the i th layer. Figure 1 gives a visualization of the model architecture:

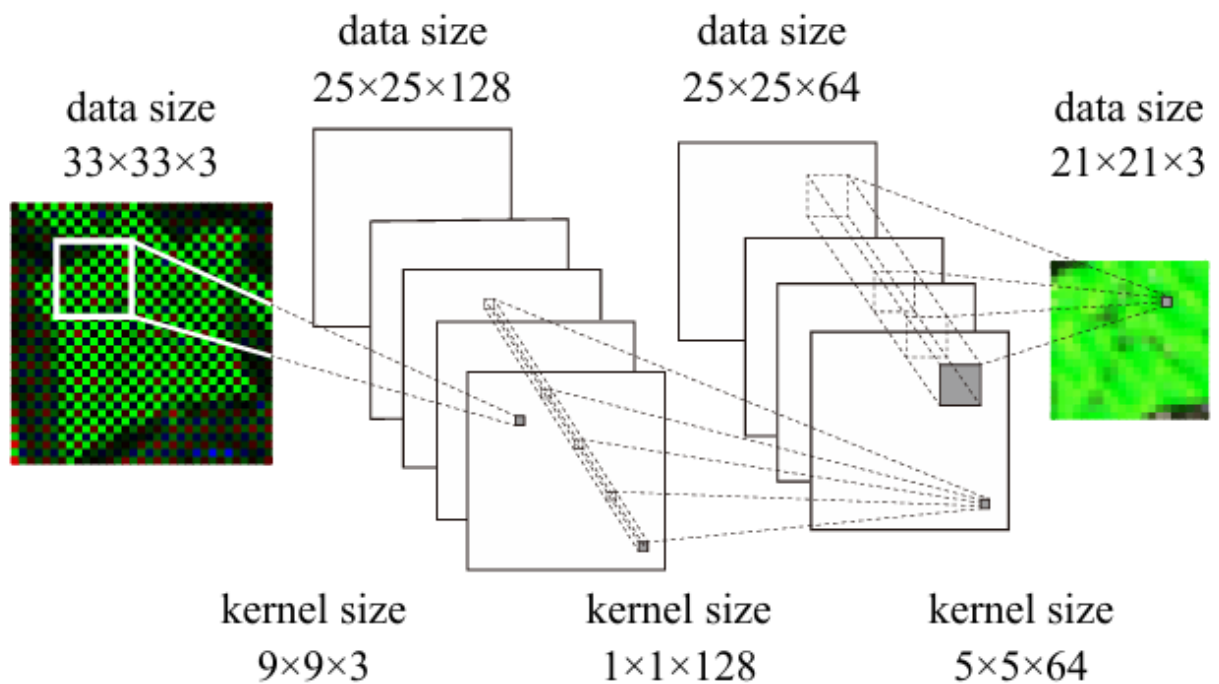


Figure 1: The architecture of DMCNN

The model is trained using Stochastic Gradient Descent (SGD) to minimize the L_2 norm (mean squared error) between the reconstructed patch and the ground truth image. During optimization, a learning rate of 1.0 is applied to the first two layers, while a lower rate of 0.1 is used for the reconstruction layer to ensure stable convergence.

Visualization of the trained network shows that it automatically learns gradient-like features with different orientations and chromatic interpolation rules. These complex spatial and

spectral relationships are often difficult for humans to design manually, but they are highly effective for reducing demosaicing artifacts. [1]

2.2 Personal implementation

This project was done as a personal effort with limited resources (e.g., hardware, technical skills, and especially time). The ultimate goal when starting this project was to obtain a working model as soon as possible, so several simplifications and adaptations were made compared to the work of Nai-Sheng Syu and their partners (hereby referred to as the original paper or original work).

While the original paper trained the model on ~3.5 million patches extracted from 500 high-quality Flickr images with extensive data augmentation, this project trained the model on ~100 000 patches extracted from 200 images collected from the BSD500 dataset [1]. While the Flickr500 dataset was manually curated to maximize color diversity and fine details, the BSD500 images are less colorful and not specifically selected for high-frequency, artifact-free color content. The BSD500 dataset was selected simply because it was easily downloaded and high-quality enough to be widely used in computer vision research [2]. In this project, after downloading BSD500, the “images” directory was used. It contains two subfolders, “test” and “train”, each of which includes 200 images. These “train” and “test” folders were used in this project as training and testing datasets, respectively.

The original paper also initialized the filters of the first and last layer with Gaussian kernels, but this was not performed in this implementation. The PyTorch used in this project was PyTorch 2.5.1, and it initializes `nn.Conv2d` layers using He uniform initialization by default. He initialization is designed to maintain stable variance propagation through ReLU-based networks and generally provides reliable convergence behavior for deep learning applications [3]. However, without the Gaussian prior, the network may require more training iterations to “discover” the smooth, interpolation-like filters that the authors found to be highly effective for demosaicing [1].

Furthermore, the original work utilized SGD as the optimization method with layer-specific learning rates; meanwhile, this project used Adam optimization with one global learning rate of 0.0001. Adam was selected primarily for implementation simplicity, faster convergence, and reduced hyperparameter tuning requirements [4]. In contrast, SGD with layer-wise learning rates may achieve superior final generalization performance and finer optimization control, but it typically requires substantially more manual tuning and longer training schedules [4]. Therefore, the use of Adam represents a pragmatic trade-off favoring rapid prototyping and training stability over strict adherence to the original optimization procedure.

To give an overview of this personal implementation, it begins by loading the BSD500 dataset using PyTorch's ImageFolder interface and converting all images into RGB tensor format. A custom DemosaicDataset class is then constructed to generate Bayer-mosaiced inputs alongside their corresponding ground-truth RGB targets. Each image is divided into overlapping 33x33 patches to allow for patch-based CNN training. Following the architecture proposed in the original work, the DMCNN model performs feature extraction, non-linear mapping, and final RGB reconstruction. Due to the use of valid convolutions without padding, the network outputs smaller 21x21 predictions from 33x33 input patches, requiring center-cropping of the target patches during training. For evaluation, a bilinear demosaicing algorithm was implemented as a classical baseline method. Full-resolution CNN inference was performed using overlapping patch-wise reconstruction and weighted stitching to recover complete RGB images. The reconstructed outputs were then compared against the ground-truth images using peak signal-to-noise ratio (PSNR). Finally, qualitative visualizations were generated to compare the mosaiced input, bilinear reconstruction, CNN reconstruction, and original ground-truth images.

3. Results and Discussions

As mentioned in section 2.2, the results of DMCNN were compared against those of the traditional handcrafted bilinear interpolation method. With 200 test images, the average PSNR for the DMCNN method is 38 dB, while the average PSNR for the Bilinear method is 28.54 dB. Here are six examples of visual comparisons, selected randomly from the 200 testing images. Figures 2, 3, 4, 5, 6, and 7 illustrate these examples:

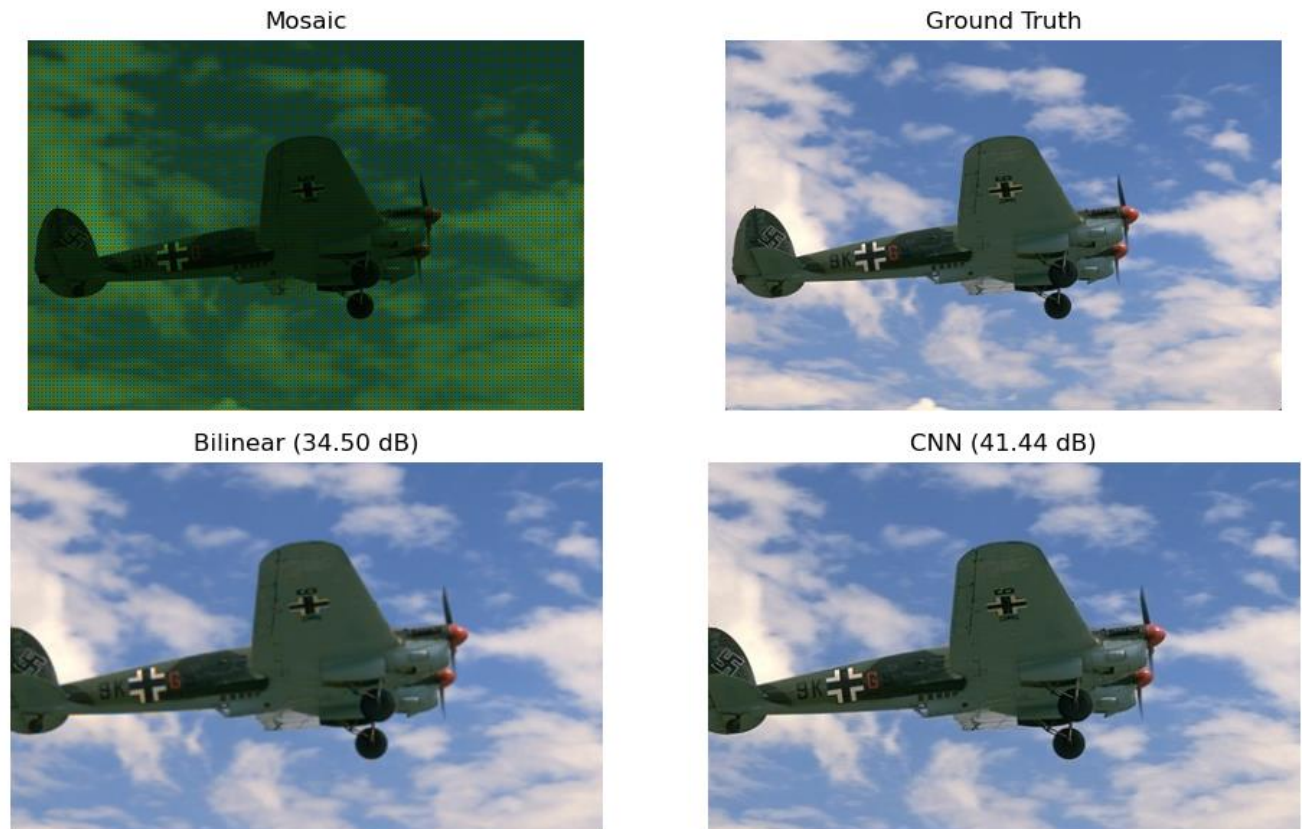


Figure 2: First comparison



Figure 3: Second comparison

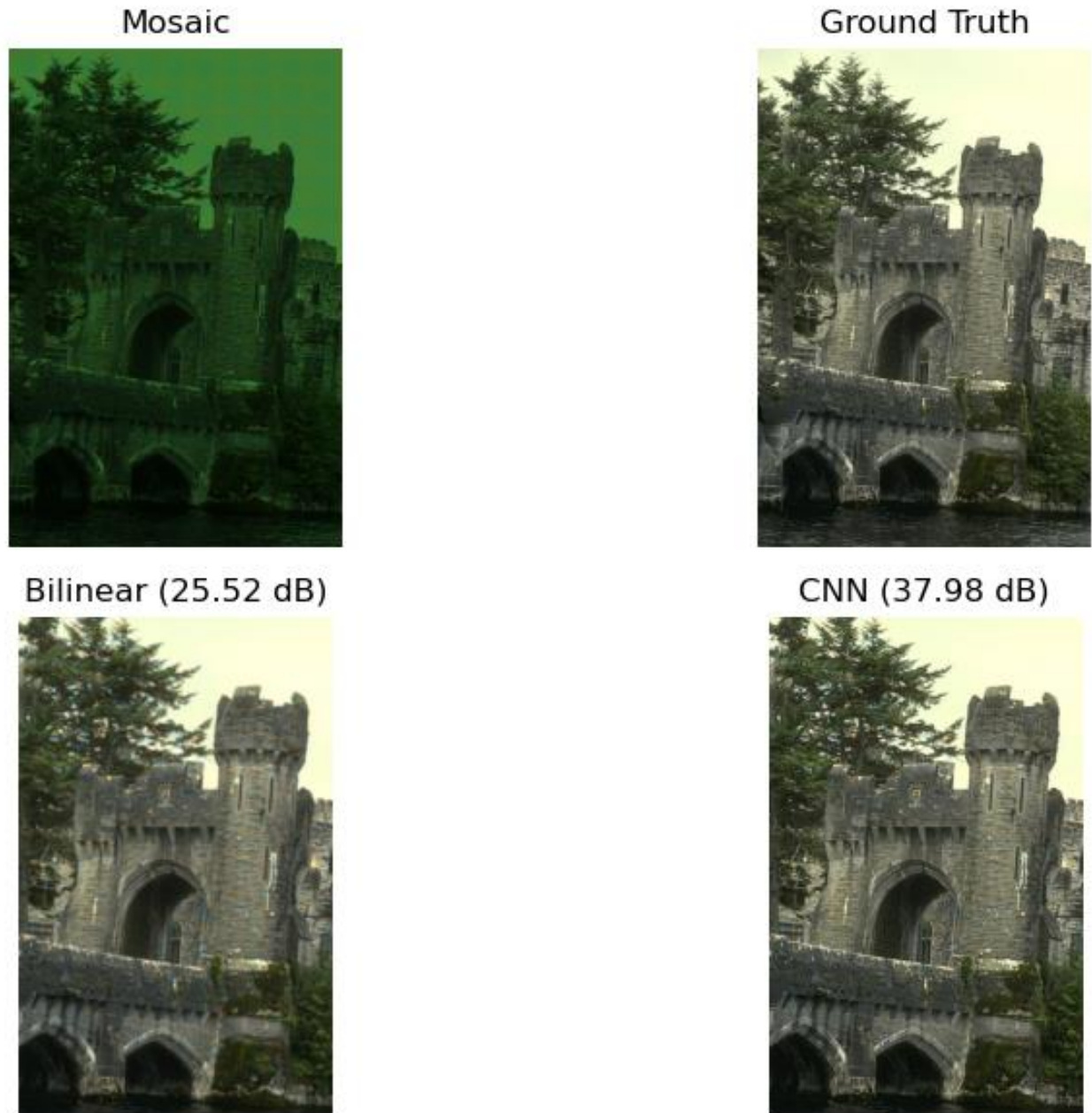


Figure 4: Third comparison

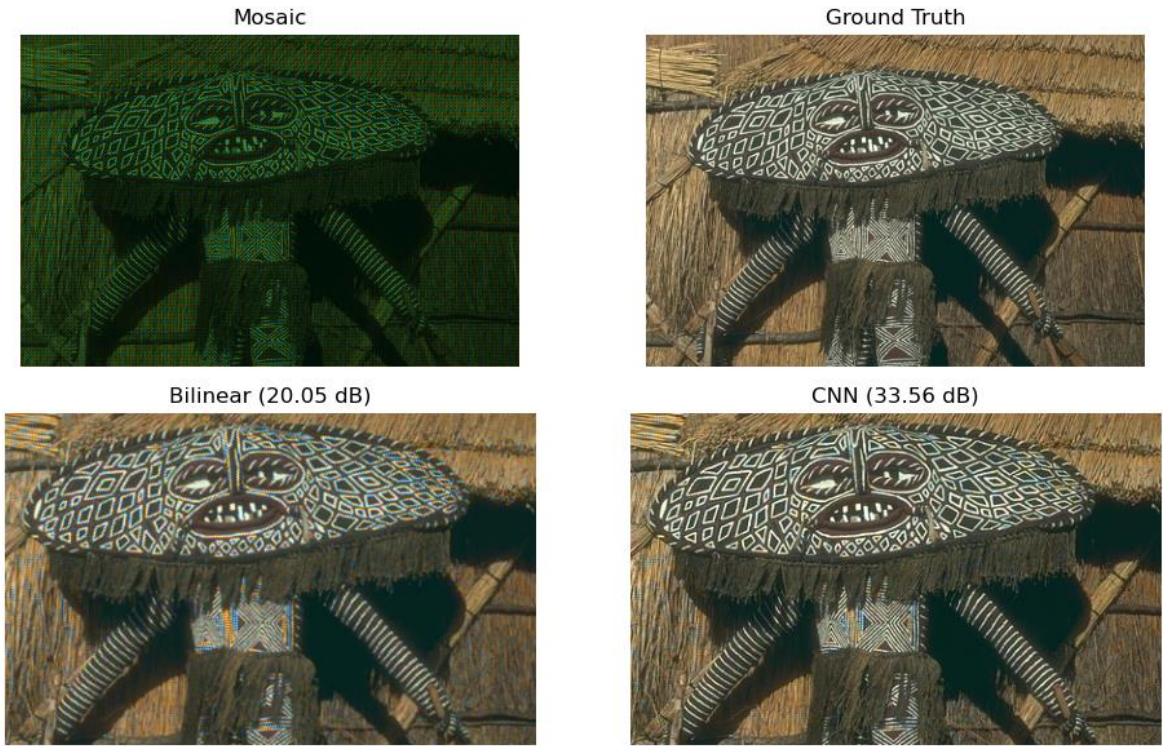


Figure 5: Fourth comparison

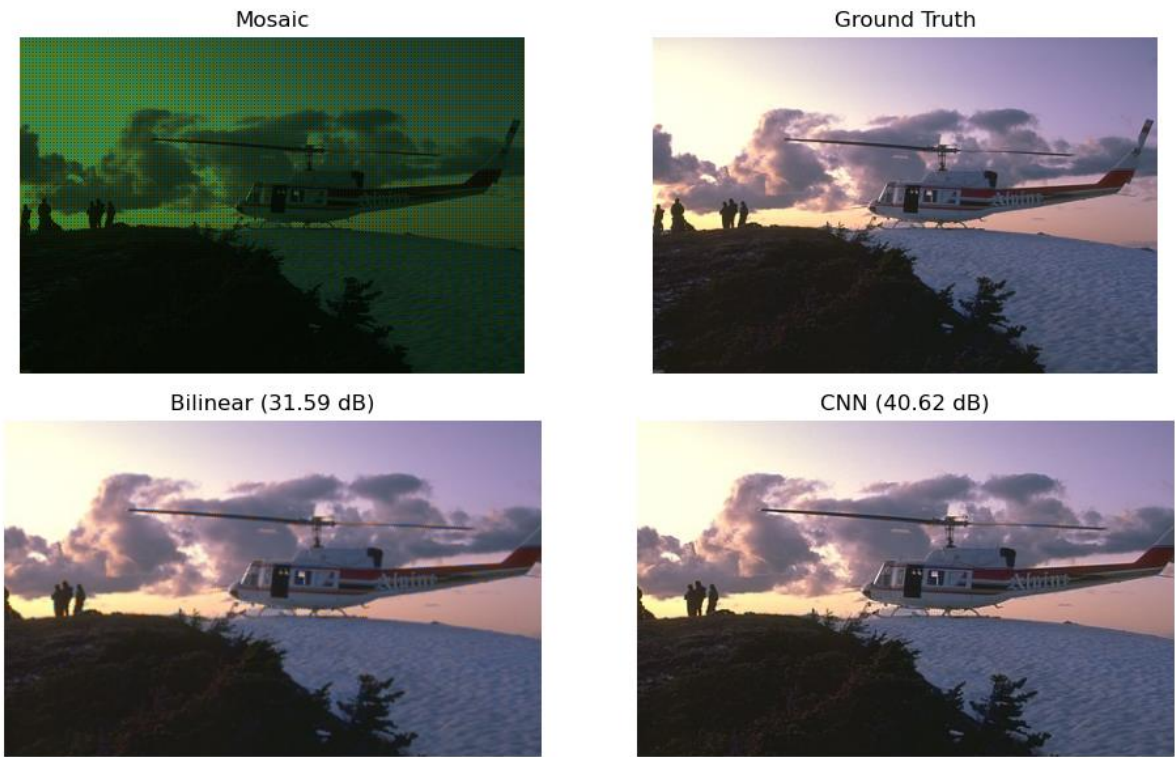


Figure 6: Fifth comparison

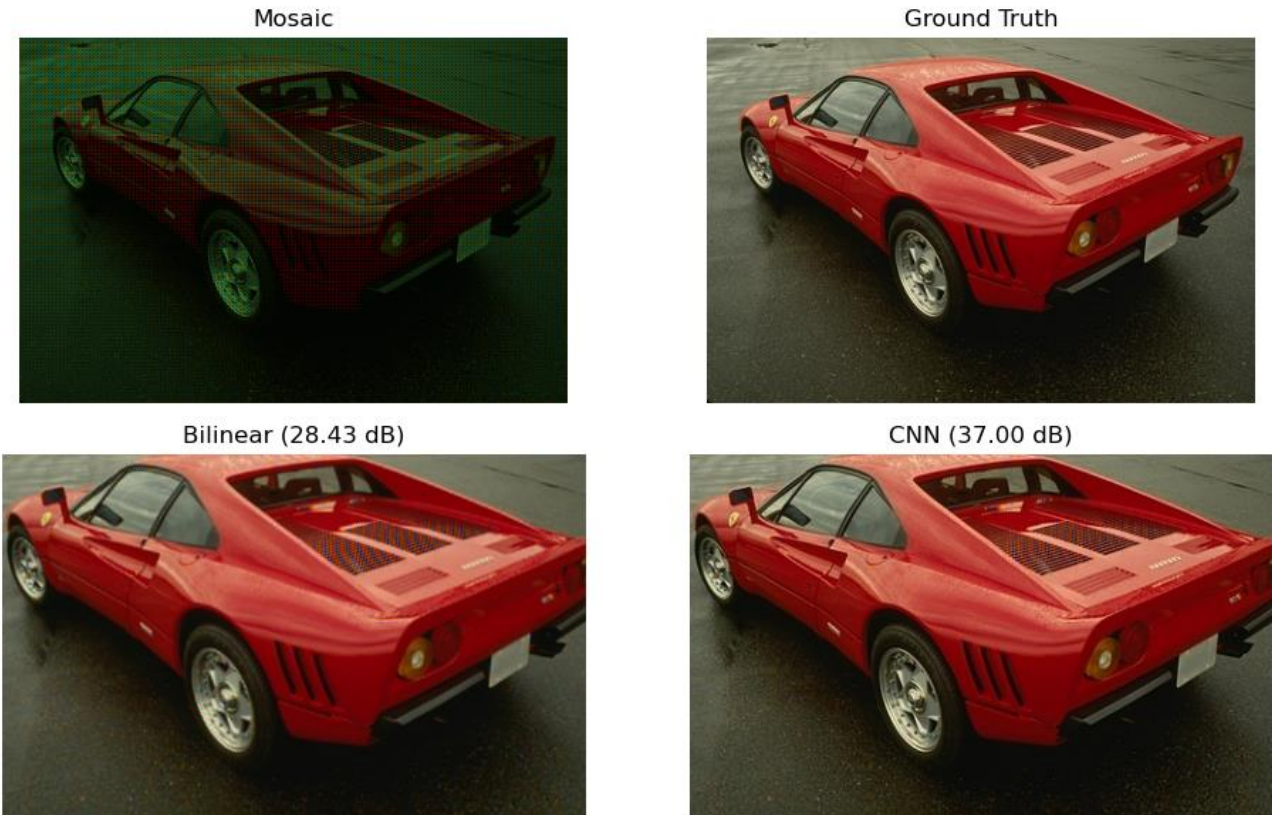


Figure 7: Sixth comparison

It can be noted easily that the bilinear results suffer from the zipper effect (can be clearly seen at the edges) more heavily than the DMCNN's results. Thus, the PSNR values of CNN images are higher than those of bilinear images.

There are some additional notes (or shortcomings) about this project's implementation. First of all, proper mosaiced images from an actual digital camera processing pipeline were not obtained, so the mosaiced input was obtained by mosaicing the colorful image. In the original image processing pipeline, these colorful images were most likely post-processed with, e.g., gamma correction or noise reduction, besides demosaicing. Thus, mosaicing the colorful image alone was not enough to obtain the proper mosaiced input.

Besides that, Nai-Sheng Syu et al. also introduced in their paper a more advanced architecture known as Very Deep DMCNN (DMCNN-VD). While the standard DMCNN is relatively shallow with only three layers, DMCNN-VD utilizes a significantly deeper structure consisting of 20 layers. Each layer in this model is composed of a convolution layer, a batch normalization layer, and a Scaled Exponential Linear Unit (SELU) activation layer. A key distinction of the DMCNN-VD model is its adoption of a residual learning strategy. Instead of attempting to map the mosaiced input directly to the full-color output, the network is

designed to learn only the "residual", the difference between the ground truth image and a baseline image generated via bilinear interpolation. Because bilinear interpolation already reconstructs smooth areas quite well, the network can focus its computational power on learning the more difficult high-frequency structures and complex artifacts. Research has shown that deeper networks are generally more powerful than shallow ones, given the same number of neurons, and the combination of depth and residual learning allows DMCNN-VD to achieve superior performance compared to both traditional methods and the standard DMCNN. [1]

4. Conclusions

This project investigated the application of deep convolutional neural networks for image demosaicing through the implementation of the DMCNN architecture proposed by Nai-Sheng Syu et al. [1]. The experimental results demonstrated that the CNN-based approach significantly outperformed the traditional bilinear interpolation method, achieving an average PSNR of 38 dB compared to 28.54 dB for bilinear demosaicing. In addition to the quantitative improvement, the visual comparisons showed that DMCNN reduced common demosaicing artifacts such as zipper effects more effectively than handcrafted interpolation methods. These findings are consistent with previous research showing that deep learning models can automatically learn complex spatial and spectral relationships that are difficult to design manually [1].

Despite the simplified implementation and limited computational resources, the model was still capable of producing high-quality image reconstructions. The use of the BSD500 dataset, Adam optimization, and default He initialization [3][4] provided a practical framework for rapid prototyping while maintaining stable convergence during training. However, several limitations remain, including the use of synthetically mosaiced images instead of raw sensor data and the omission of more advanced architectures such as DMCNN-VD. Future work could therefore explore deeper residual-based networks, larger training datasets, and more realistic camera processing pipelines to further improve reconstruction quality and generalization performance. Additionally, the original paper also proposed that CNN-based methods can be adopted for demosaicing with CFA designs other than the Bayer one. They also presented the first data-driven method for joint optimization of the CFA design and the demosaicing method. Thus, these are some potential future research directions that one could delve into.

References

[1]: Syu, Nai-Sheng & Chen, Yu Sheng & Chuang, Yung-Yu. (2018). Learning Deep Convolutional Networks for Demosaicing. 10.48550/arXiv.1802.03769.

[2]: Berkeley Segmentation Dataset 500 (BSDS500). Kaggle. Access at: <https://www.kaggle.com/datasets/balraj98/berkeley-segmentation-dataset-500-bsds500>

[3]: Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. 2015, IEEE International Conference on Computer Vision (ICCV). Access: https://www.researchgate.net/publication/272027131_Delving_Deep_into_Rectifiers_Surpassing_Human-Level_Performance_on_ImageNet_Classification

[4]: Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. 2014. Access: <https://arxiv.org/abs/1412.6980>